This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

Surrogate Management Strategies for Accelerating Parallel Algorithm Portfolio Construction

Grzegorz Zakrzewski, Xin Yao, *Fellow, IEEE*, and Jacek Mańdziuk, *Senior Member, IEEE*

***Abstract*—Many algorithms require careful parameterization, whose choice significantly impacts performance, with no single configuration excelling across all problem types. Automatic Algorithm Configuration provides a systematic approach to parameter exploration, enabling the identification of problem-specific optimal configurations. Automatic Construction of Parallel Portfolio (ACPP) extends this idea by configuring and running multiple solvers simultaneously in parallel, exploiting their complementary strengths for improved generalization. Existing ACPP approaches treat construction time as a fixed budget and make only partial use of the performance data collected during component solver evaluation. To address this limitation, our solution relies on using the data generated during portfolio construction to train surrogate models that replace costly evaluations to reduce computational overhead. Furthermore, to identify effective surrogate models, we compare various machine learning approaches, including applications of survival analysis techniques for handling censored runtime data. We evaluate these models' predictive capabilities using data generated by the LKH solver on the Traveling Salesperson Problem instances. Finally, we propose several surrogate model management strategies and integrate them into existing portfolio construction approaches. Our results demonstrate that survival-based models achieve superior predictive performance compared to standard machine learning approaches. The proposed surrogate model management strategies offer significant savings in portfolio construction time while maintaining acceptable performance on unseen test instances.**

***Index Terms*—parallel algorithm portfolio, automatic algorithm configuration, surrogate models, survival analysis, Traveling Salesperson Problem.**

I. Introduction

ALGORITHMIC solvers for computationally hard problems require careful parameterization whose settings can significantly impact their performance. However, each algorithm configuration typically excels only on specific problem types, with no single configuration consistently outperforming others across all cases. This limitation aligns with the No Free Lunch theorem [1], which states that no algorithm can be optimal across all problem types.

While high configurability allows algorithms to be tailored to specific problem instances, it also introduces challenges. Users often struggle to identify effective parameter settings when applying algorithms to particular problem instances. Manual configuration is typically ad-hoc, exploring only a limited set of configurations and often resulting in suboptimal performance. Finding optimal configurations for given instances requires extensive experimentation, which is often slow, resource-intensive, and demands a deep understanding of both the algorithm and problem domain. Furthermore, many algorithms exhibit complex parameter interactions, making the tuning process even more challenging and time-consuming.

The challenges of manual parameter tuning have led to the development of Automatic Algorithm Configuration (AAC) approaches [2], [3]. AAC optimizes the algorithm performance by automating the process of parameter settings selection [4]. Instead of relying on expert human intervention, AAC systematically explores various configurations to identify those that achieve optimal results on specific problem instances. This process involves iterating through configurations, testing them on representative problem instances, and selecting the configuration with the highest performance.

AAC has shown considerable advantage, particularly with configurators such as ParamILS [5], GGA [6], *irace* [7], and SMAC [8]. These tools enable researchers to explore large parameter spaces and develop high-performing solvers tailored to specific applications. However, a configuration identified by AAC may not generalize well to the entire problem distribution.

In recent years, the increased computational power has significantly impacted computationally hard problem solvers and AAC approaches [9]. To exploit this increased power, researchers have focused on developing *parallel solvers*. However, designing parallel solvers is complex, often requiring near-complete redesign of existing sequential solvers and considerable domain knowledge [10]. Consequently, an alternative approach has gained popularity: executing multiple instances of sequential solvers simultaneously.

The concept that combines AAC and parallel execution is the Parallel Algorithm Portfolio (PAP) [11], [12]. PAPs solve computationally challenging problem instances by running multiple solvers in parallel. This approach leverages solver

Xin Yao's work was partially supported by an internal grant from Lingnan University. Grzegorz Zakrzewski and Jacek Mańdziuk were partially supported by the National Science Centre, Poland, grant number 2023/49/B/ST6/01404. The authors gratefully acknowledge the funding support by program "Excellence initiative-research university" for the AGH University of Krakow as well as the ARTIQ project: UMO-2021/01/2/ST6/00004 and ARTIQ/0004/2021. The research was carried out with the support of the HPC Center of the Faculty of Mathematics and Information Science, Warsaw University of Technology. (Corresponding authors: Grzegorz Zakrzewski, Xin Yao).

Grzegorz Zakrzewski is with the Faculty of Mathematics and Information Science, Warsaw University of Technology, Warsaw, Poland, and also with the Institute of Meteorology and Water Management – National Research Institute, Warsaw, Poland (e-mail: grzegorz.zakrzewski.dokt@pw.edu.pl).

Xin Yao is with the School of Data Science, Lingnan University, Hong Kong, China (e-mail: xinyao@ln.edu.hk).

Jacek Mańdziuk is with the Faculty of Mathematics and Information Science, Warsaw University of Technology, Warsaw, Poland, and also with the Center of Excellence in Artificial Intelligence, AGH University of Krakow, Krakow, Poland, and also with the Faculty of Computer Science, AGH University of Krakow, Krakow, Poland (e-mail: jacek.mandziuk@pw.edu.pl).

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

diversity, and a well-constructed portfolio generalizes across a wide range of instances. PAPs are particularly well-suited to multi-core systems, where solvers can operate independently and simultaneously [13].

The Automatic Construction of Parallel Portfolios (ACPP) involves selecting and tuning solvers to create portfolios that perform well across a given problem distribution [14]. This is an offline process that requires representative training instances and a diverse configuration space of potential solvers. ACPP applies AAC techniques to identify and optimize portfolios based on their performance on the training set. The goal is to build portfolios that generalize effectively to new problem instances from the same distribution as the training instances. Once constructed, the portfolio enters an online phase in which solvers run in parallel on each new problem instance. If the objective is runtime minimisation, the portfolio result is the outcome of the fastest component solver, and other component solvers are terminated as soon as the first one returns a result.

Notable ACPP methods include GLOBAL [15], PARHYDRA [15], [16], CLUSTERING [15], [17], and PCIT [18]. Methods such as GAST [19] and CEPS [20] address the challenge of limited training samples. However, these papers do not examine the portfolio construction time or identify its primary contributing factors. Construction time is typically treated as a fixed constant budget across the experiments. Furthermore, during the ACPP process, the algorithms are executed multiple times on the training instances, generating substantial algorithm runtime data. Current ACPP approaches use component solver performance on problem instances to guide AAC, without further leveraging this performance data.

A natural way to use this generated performance data is to train a machine learning (ML) model that maps features of the algorithms and problem instances to the expected algorithm runtime. The model can then be used to replace part of the evaluations during the portfolio construction process by estimating the results and thus accelerating the process of ACPP. This approach is well-established in the related areas such as Surrogate-Assisted Evolutionary Algorithms (SAEAs) [21]–[23] and Automatic Algorithm Selection [24]–[27], but has been scarcely applied in the context of algorithm portfolios.

Surrogate models are also used internally by model-based AAC procedures such as SMAC3 [28]. However, such surrogates serve a different purpose from the one we propose: the AAC surrogate guides the search through the configuration space by predicting the quality of candidate configurations, whereas our surrogate replaces individual evaluations of solver configurations on specific problem instances during the portfolio construction process.

Additionally, due to potentially extremely long training, the algorithm runtime data is usually generated under time constraints, meaning that not all algorithms are run to completion on all instances. Hence, the collected data typically comprises censored information, as the true runtime of timed-out algorithms remains unknown [29], [30]. Although in the literature ML models have been trained on algorithm runtime data [29]–[33], there is no systematic comparison of which model best suits this type of data.

A. Contribution

Our recent conference paper [34] presented an initial study examining two approaches: (i) training a surrogate model on data generated during portfolio configuration to replace part of the evaluations and reduce computational cost, and (ii) applying portfolios trained on smaller problem instances to solve more complex instances, to address the scalability aspect. Building on these initial results, this paper extends the surrogate model approach by identifying the most appropriate ML model and model management strategy in a thorough and systematic way.

The main contribution of this paper is six-fold:

(i) We define three desirable properties for ML models used as surrogates in ACPP: high predictive quality, low data requirements, and the ability to handle censored runtimes;
(ii) We evaluate six standard ML models, commonly used for surrogate modeling or algorithm runtime prediction, and examine six survival analysis models and techniques;
(iii) We empirically compare these models using algorithm runtime data generated by the LKH solver [35], [36] on a wide selection of the Travelling Salesperson Problem (TSP) instances;
(iv) We propose five surrogate model management strategies for integration into the PAP construction process. Such a strategy defines rules for when to use a surrogate model to predict algorithm runtime rather than executing the algorithm directly;
(v) We incorporate these strategies into the PARHYDRA portfolio construction approach, showing their high effectiveness on TSP instances, and assessing their impact on both the overall construction time and portfolio performance.
(vi) We verify the generality of our results through four additional experiments involving (1) the well-known TSPLib [37] benchmark (2) portfolios containing four solvers (instead of two), (3) an alternative AAC procedure, and (4) a different problem domain, namely continuous optimization. These additional experiments are reported in Section X of the supplementary material.

Our empirical results show that using appropriate surrogate models and management strategies can substantially reduce portfolio construction time with minimal performance deterioration, making them practical for a wide range of problem-solving scenarios and contributing to lower computational and energy costs associated with large-scale algorithm configuration.

II. Related Work

A. Learning to optimize

Learning to optimize (L2O) [38] is the umbrella term for methodologies that balance algorithm configuration time with solution quality by leveraging knowledge from training problem instances. L2O comprises three main fields: Automatic Algorithm Selection, Automatic Algorithm Configuration, and Algorithm Portfolios.

Automatic Algorithm Selection [14], [24]–[27] uses predictive models to select the best-performing algorithm for a

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

given problem instance. Often, this process relies on instance-specific features that must be informative, computationally inexpensive, and applicable across diverse instances. An ML model is trained to map instance features to solver performance. The model enables the selection of the most suitable algorithm for a given problem instance with minimal computational effort. The model applies regression to estimate solver performance or classification to predict the optimal solver directly.

Automatic Algorithm Configuration [2], [3], [18]–[20] determines optimal parameter settings for the algorithms by systematically testing different parameter combinations on specific problem instances. AAC approaches require careful consideration of common pitfalls, including proper handling of algorithm stochasticity and prevention from overfitting to training instances [39]. Reliable outcomes demand rigorous experimental design incorporating multiple random seeds and representative instance sets.

Algorithm Portfolios [11], [12], [15]–[20], [40]–[42] address the performance variability of algorithms when solving computationally hard problems. Rather than relying on a single solver, Algorithm Portfolios combine multiple algorithms or algorithm configurations to exploit performance diversity across problem instances.

Several approaches for ACPP have been developed. GLOBAL [15] optimizes all solvers together as one configuration task, but its configuration space grows exponentially with the portfolio size, making it impractical for larger portfolios. HYDRA [16] adds configurations iteratively, targeting instances where the current portfolio performs poorly. PARHYDRA [15] extends HYDRA by configuring solvers in parallel during each iteration to find complementary configurations. ISAC [17] clusters training instances by their features and optimizes configurations for each cluster. CLUSTERING [15] adapts ISAC to PAPs by running configuration processes in parallel and selecting the best portfolio. PCIT [18] improves CLUSTERING by migrating instances between clusters during configuration based on performance data. GAST [19] uses an adversarial framework for cases with limited or biased training data, generating problem instances while constructing portfolios. CEPS [20] uses co-evolution to develop solver configurations and problem instances as separate populations, improving performance when training data is scarce.

B. Surrogate modeling

To our knowledge, only PCIT [18] explicitly employs surrogate models to transfer instances between clusters. Instead of conducting additional costly evaluations, it builds performance models to identify poorly solved instances and reassigns them to more suitable solvers.

Surrogate models have been also extensively studied outside the context of ACPP [22]. Hutter et al. [31] provided a comprehensive review of runtime prediction methods, showing that Random Forests consistently outperformed other models in this task.

Eggensperger et al. [32] proposed a neural network-based approach that estimates runtime distributions rather than mean runtimes alone for SAT solvers. To address the computational cost of evaluating algorithm configurators, Eggensperger et al. [33] introduced surrogate scenarios – computationally efficient approximations that predict algorithm performance.

Surrogate models are central to per-instance algorithm selection. Kerschke et al. [43] demonstrated strong performance complementarity among inexact TSP solvers, where different solvers excel on different instances. A comprehensive survey of per-instance algorithm selection for combinatorial and continuous optimization by Kerschke et al. [25] emphasizes the importance of informative and computationally efficient instance features. Tanabe [26] conducted a systematic benchmarking study of algorithm selection systems for black-box numerical optimization, emphasizing that the choice of evaluation metric can substantially affect performance assessment.

C. Modeling with survival analysis

Survival analysis is a statistical framework originally developed to model the time until an event occurs, such as patient death. In many cases, the exact event time is unknown because it occurs after the observation period ends; such data are referred to as right-censored [44]. An analogous situation arises in algorithm runtime prediction: algorithms may not find a solution within predefined time limits, leading to censored runtime data. This issue also affects algorithm portfolios, where not all solvers run to completion. Survival analysis naturally accommodates censored observations, making it well-suited for modeling algorithm runtimes under time constraints.

We now shortly introduce survival analysis terminology and summarize six survival-based methods used in the experiments in Section IV.

In survival analysis, the data have the form:

$$\mathcal{D} = \{(x_n, y_n, \delta_n)\}_{n=1}^{N}, \tag{1}$$

where $x_n \in \mathbb{R}^p$ is a p-dimensional vector of covariates, $y_n \in \mathbb{R}_+$ is the observed time (either event time or censoring time), and $\delta_n \in \{0, 1\}$ is a binary indicator denoting whether the observation is uncensored ($\delta_n = 1$) or right-censored ($\delta_n = 0$). If $\delta_n = 1$, then y_n corresponds to the actual event time T_n (e.g., time required by the algorithm to solve an instance). If $\delta_n = 0$, then y_n is a lower bound on T_n, equal to a predefined cut-off time C_n.

The aim is to build a model that, given covariates x, accurately predicts the unknown survival time T. During training, only partial information is available – namely, y and δ. Some survival analysis models aim to predict T directly, while others define a probabilistic model to capture this partial information. In probabilistic approaches, T is treated as a random variable, modeled through the survival function

$$S(t \mid x) = \mathbb{P}(T \geq t \mid x), \tag{2}$$

which represents the probability that the event of interest does not occur before time t, given covariates x.

Another relevant notion is the hazard function

$$h(t \mid x) = \lim_{\Delta t \to 0} \frac{\mathbb{P}(t \leq T < t + \Delta t \mid T \geq t, x)}{\Delta t}, \tag{3}$$

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

which can be interpreted as the instantaneous rate at which events occur at time t, conditional on survival up to time t. These functions are related; knowing one allows derivation of the others. In particular

$$S(t \mid x) = \exp\left(-\int_0^t h(u \mid x)\, du\right), \tag{4}$$

where the integral is called the cumulative hazard function:

$$H(t \mid x) = \int_0^t h(u \mid x)\, du. \tag{5}$$

Rather than in probabilistic predictions, in this work we are interested in producing a point estimate of the algorithm's runtime. One approach is to compute the expected value of the random variable T:

$$\mathbb{E}[T] = \int_0^\infty S(t \mid x)\, dt. \tag{6}$$

Tornede et al. [29] argue that the expected value may be suboptimal in some applications. They propose the use of an additional risk function $\mathcal{L}$ to transform the runtime before computing the expected value: $\mathbb{E}[\mathcal{L}(T)]$. Following this idea, we consider four risk functions:

- linear: $\mathcal{L}(t) = t$;
- polynomial: $\mathcal{L}_\alpha(t) = t^\alpha$;
- exponentials: $\mathcal{L}_{\alpha,\beta}(t) = \min\left\{-\alpha \log\left(1 - \frac{t}{t_{\max}}\right), \beta\right\}$;
- PAR10: $\mathcal{L}(t) = \text{PAR10}(t) = \begin{cases} t, & \text{if } t \leq C, \\ 10 \cdot C, & \text{otherwise,} \end{cases}$

where α and β are tunable parameters, $t_{\max}$ is the maximum observed runtime in the training set, and C is a predefined cut-off time used to censor the algorithm run. The choice of the risk function and its parameters are treated as a hyperparameter optimization problem.

Model 1: Cox proportional hazards model [45] is defined as:

$$h(t \mid x) = h_0(t) \cdot \exp\left(\sum_{i=1}^p x^\top \beta\right), \tag{7}$$

where $h_0(t)$ is the baseline hazard function, which can be estimated using Breslow's estimator [46]. The coefficient vector β is estimated by maximizing the partial likelihood:

$$\ell(\beta) = \sum_i \delta_i \left(x_i^\top \beta - \log \sum_{j \in \mathcal{R}(y_i)} \exp(x_j^\top \beta)\right), \tag{8}$$

where $\mathcal{R}(y_i)$ denotes the set of solvers still running at time y_i.

Model 2: Gradient Boosting with Cox's partial likelihood loss [47]–[49] extends the Cox model by replacing the linear predictor $x^\top \beta$ with a non-linear additive function $f(x)$ trained via gradient boosting.

Model 3: Random Survival Forests [50] are ensemble models that directly estimate the cumulative hazard function $H(t \mid x)$, from which survival probabilities can be derived.

Model 4: XGBoost with the Accelerated Failure Time loss function [51], unlike the three aforementioned methods, directly estimates logarithmized runtime rather than hazard or survival functions.

Model 5: A neural network architecture proposed by Eggensperger et al. [52] follows a similar approach. This network estimates both mean $\hat{\mu}_i$ and variance $\hat{\sigma}_i^2$ and is **trained with a Tobit loss function**, which handles censored and uncensored observations separately:

$$-\log \mathcal{L}\left(\left(\hat{\mu}_i, \hat{\sigma}_i^2\right)_{i=1}^n \mid \mathcal{D}\right) = -\sum_{i=1}^n \log\left(\phi\left(\frac{c_i - \hat{\mu}_i}{\hat{\sigma}_i^2}\right)^{\delta_i}\left(1 - \Phi\left(\frac{c_i - \hat{\mu}_i}{\hat{\sigma}_i^2}\right)^{1-\delta_i}\right)\right) \tag{9}$$

Here ϕ and Φ denote the standard normal density and cumulative distribution functions, respectively.

Model 6: The Schmee & Hahn (S&H) imputation method [33], [53], [54] is the last model considered. The method can work with any model capable of estimating standard deviation alongside the mean for a given observation. In the experiments, we used Quantile Random Forest [55]. The method operates as follows: first, a model is trained on uncensored data only. Then, for every censored observation i in the training dataset, the mean $\hat{\mu}_i$ and variance $\hat{\sigma}_i^2$ are estimated using the model. The censored runtime y_i is imputed with the mean of the truncated normal distribution $\text{TN}(\hat{\mu}_i, \hat{\sigma}_i^2, y_i, \infty)$. The process of training the model and imputing originally censored runtimes can be repeated for multiple iterations. After the first iteration, the model is trained on all observations, both uncensored and imputed.

Gagliolo and Legrand [56] first applied survival analysis methods to algorithm runtime prediction. Hutter et al. [54] extended the S&H method, and in subsequent work [31], this extended S&H method was evaluated on datasets with substantial proportions of censored runs and outperformed models that ignored censoring or discarded censored instances.

Tornede et al. [29] applied survival analysis techniques to algorithm selection, developing a framework based on Random Survival Forests and evaluating it using datasets from ASlib [57], where some datasets contained over 70% of censored observations. This work was later extended by Margraf et al. [58] to algorithm scheduling.

De Souza et al. [59] introduced capping strategies for tuning optimization algorithms where performance is measured by solution quality rather than runtime, leveraging earlier evaluations to terminate poor configurations prematurely. Similarly, Tornede et al. [60] investigated online algorithm selection under censored feedback from runtime timeouts.

D. Surrogate-Assisted Evolutionary Algorithms

Surrogate models have been extensively applied in evolutionary algorithms. Jin [21] provided a comprehensive overview of surrogate-assisted evolutionary computation, covering development, management strategies, and applications to computationally expensive optimization problems. Tong et al. [22] proposed a taxonomy categorizing surrogate models into absolute and relative fitness models. Both works emphasize balanced approaches combining surrogate estimations with real evaluations to ensure solution quality, cautioning against exclusive reliance on surrogate predictions that may prevent discovery of globally optimal solutions.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

III. Methodology

This section presents the PAPs and methods we introduce to accelerate the PAP construction process.

Our approach builds upon insights from initial experiments reported in our previous work [34], where surrogate modeling was successfully applied within the GLOBAL portfolio construction approach. However, the cut-off time – which determines when to terminate evaluation of unpromising algorithm configurations – emerged as a critical factor. Cut-off times create censored runtime observations, fundamentally skewing the distribution of algorithm runtimes toward lower values. ML models trained on such censored data inherit this bias, resulting in systematically inaccurate predictions that can mislead the portfolio construction process.

These findings highlighted the need for surrogate models that provide accurate predictions while handling censored runs in training data, along with effective management strategies that maximize surrogate model utilization while minimizing adverse effects on portfolio performance. Based on these conclusions, we developed the methodology for surrogate models described in Section III-C and surrogate model management strategies presented in Section III-D. Section III-A describing the general ACPP framework, and Section III-B presenting the PARHYDRA portfolio construction method are primarily based on existing literature [15], [16], and we introduced minor modifications only.

A. Automatic Construction of Parallel Portfolio

The general ACPP framework builds a portfolio of component solvers, where each solver represents a distinct algorithm configuration. We denote a PAP as Θ, consisting of K component solvers $\theta_1, \ldots, \theta_K$. Each component solver θ_i belongs to an algorithm configuration space $\mathcal{C}_i$, such that $\theta_i \in \mathcal{C}_i$. The complete portfolio configuration space $\mathcal{C}$ is the Cartesian product of individual component configuration spaces: $\mathcal{C} = \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K$.

Portfolio construction requires a training sample S_T, which must be a representative subset of the problem distribution. The performance of portfolio Θ on problem instance s is determined by executing all component solvers independently and selecting the best result: $f(s, \Theta) = \min\{f(s, \theta_1), \ldots, f(s, \theta_K)\}$ (without loss of generality, we assume that S is a minimization problem). The objective during portfolio configuration is to optimize performance across the training sample by minimizing $\bar{f}(\Theta) = \frac{1}{|S_T|} \sum_{s \in S_T} f(s, \Theta)$.

We introduce a surrogate model $\hat{f}(s, \theta)$ that estimates the performance for specific algorithm configurations on given problem instances, i.e., can substitute computationally expensive algorithm evaluations. A surrogate model management strategy determines when to use surrogate predictions instead of actual evaluations. Sections III-C and III-D provide detailed descriptions of the proposed surrogate model approach.

ACPP employs an AAC procedure that iteratively improves portfolio performance on S_T. The procedure generates new configurations that are added to the portfolio or replace the weakest-performing configurations. The new configurations are evaluated on S_T, and the AAC uses performance feedback to improve subsequent configuration proposals. This iterative process continues until the configuration budget is exhausted. Algorithm 1 presents the basic ACPP framework.

The AAC procedure can utilize an internal surrogate model. It is important to distinguish this model from our surrogate $\hat{f}(s, \theta)$. The AAC surrogate operates at the iteration level and aggregated results (Algorithm 1, lines 3 and 11–12), whereas our surrogate replaces individual solver evaluations on specific problem instances (Algorithm 1, line 7).

Algorithm 1 Basic framework of ACPP

Require: AAC: Automatic Algorithm Configuration Procedure
Require: K: number of solvers in the portfolio
Require: $\mathcal{C} = \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K$: portfolio configuration space
Require: S_T: training sample
Require: f: performance measure
Require: $\hat{f}$: surrogate model
1: Initialize portfolio $\Theta = \{\theta_1, \ldots, \theta_K\}$ with initial configurations
2: **while** budget available **do**
3: AAC proposes new configurations Θ^* to replace a subset of Θ
4: Update portfolio Θ with proposed configurations from Θ^*
5: **for** each training instance $s \in S_T$ **do**
6: **for** each component solver $\theta_i \in \Theta$ **do**
7: Record performance $f(s, \theta_i)$ or $\hat{f}(s, \theta_i)$ according to surrogate model management strategy
8: **end for**
9: Calculate portfolio performance $f(s, \Theta) = \min_{i=1}^{K} f(s, \theta_i)$
10: **end for**
11: Calculate average performance $\bar{f}(\Theta) = \frac{1}{|S_T|} \sum_{s \in S_T} f(s, \Theta)$
12: Provide average performance $\bar{f}(\Theta)$ to the AAC procedure
13: **if** surrogate model is used **then**
14: Retrain surrogate model $\hat{f}$ with new evaluation data
15: **end if**
16: **end while**
17: **return** constructed portfolio Θ

B. PARHYDRA approach for ACPP

PARHYDRA [15] starts with a single configuration and expands the portfolio up to K solvers by targeting instances where the existing portfolio performs poorly.

When searching for the ith solver, PARHYDRA performs $MaxAttempt$ parallel configuration attempts to find a new solver. Each attempt uses the same starting configuration but explores different parts of the configuration space. The best configuration found is added to the current portfolio as a new solver.

After adding the solver to the portfolio, all candidate configurations from the current round are evaluated for their marginal contribution – that is, how much each of them would improve the performance of the extended portfolio (with the newly added solver) if it were added as an additional solver. The configuration with the highest marginal contribution is selected as the starting point for the next round of configuration. This strategy helps to diversify the portfolio by guiding the search toward regions in the configuration space that complement the already selected solvers.

We adapt the PARHYDRA approach for our ACPP framework by modifying the portfolio construction budget from a CPU time limit to an iteration limit. This modification enables

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

direct measurement of construction time savings achieved by the proposed acceleration methods. Algorithm 2 presents the detailed steps of this approach. This modified PARHYDRA approach is used in the experiments presented in Section IV-B.

Algorithm 2 PARHYDRA approach for ACPP

Require: AAC: Automatic Algorithm Configuration Procedure
Require: $Budget$ a budget of iterations
Require: $MaxAttempt$ a number of attempts
Require: K: number of solvers in the portfolio
Require: $\mathcal{C} = \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K$: portfolio configuration space
Require: S_T: training sample
Require: f: performance measure
1: Initialize empty portfolio $\Theta = \{\}$
2: **for** $i \in [K]$ **do**
3: **for** $attempt \in [MaxAttempt]$ **do**
4: **for** $iter \in [Budget]$ **do**
5: AAC proposes new configuration θ_i^* from $\mathcal{C}_i$
6: Replace configuration θ_i with θ_i^*
7: Calculate average performance $\bar{f}(\Theta)$ (as in lines 5–11 in Algorithm 1) and provide it to the AAC procedure
8: **end for**
9: Record attempt solver $\theta_i^{attempt}$
10: **end for**
11: Add the best attempt solver θ_i^{best} to Θ
12: Set the attempt solver $\theta_i^{attempt}$ that gives the best performance $\bar{f}(\Theta \cup \{\theta_i^{attempt}\})$ as the starting point for AAC search for the configuration of the next component solver θ_{i+1}
13: **end for**
14: **return** constructed portfolio Θ

C. Machine learning models for surrogate-assisted ACPP

Surrogate models offer an effective framework for leveraging performance data generated during portfolio construction. During the ACPP runtime, the AAC procedure evaluates numerous configurations on training instances, producing triplets (component solver, problem instance, performance). This data can be used to train a surrogate model that estimates the performance function $\hat{f}(s, \theta)$, using solver configuration θ and problem instance s as inputs and solver performance as the target. The surrogate model accelerates portfolio construction by providing immediate performance estimates, replacing costly evaluations.

The implementation of surrogate models in this context presents several challenges. First, training accurate surrogate models requires descriptive features for both solvers and problem instances. While solver features can be derived from parameter values, obtaining informative instance features may be challenging, especially for new or less-studied problem types. Second, the available training data depends on portfolio size and construction budget. Since runtime data accumulates during portfolio construction, training data is initially scarce, preventing surrogate model deployment right from the first iteration until sufficient training data is collected. Finally, determining the optimal balance between the real and surrogate-based evaluations requires careful consideration. This issue of selecting appropriate surrogate model management strategies is examined in Section III-D.

Given these challenges, we point out three desirable properties for effective surrogate models in PAP construction. First, we expect high-quality predictions that correctly guide the algorithm configuration procedure. Second, the surrogate model must have minimal data requirements – each evaluation of the true solver on a training problem instance produces one observation in the model training dataset, and we want to keep the number of true evaluations low. The third property concerns handling of censored data, and will be addressed in detail after discussing standard ML approaches.

Based on previous works in Algorithm Runtime Prediction (ARP), Automatic Algorithm Selection (AAS), and SAEAs, we identify several commonly used ML models as candidates for surrogate models in the PAP construction process. Ridge Regression (introduced in [61]) is a simple model applied in ARP contexts [31], [62], [63]. Polynomial Regression (see e.g., [64]) has been used in SAEAs [22], [65]. Support Vector Regression [66] has been applied to both AAS [43], [57] and SAEAs [22], [65]. Gaussian Process Regression [67] with Radial Basis Function has been used in ARP [31] and SAEAs [22], [65]. Random Forest Regressor [68] has been applied in ARP [31], [33], [69] and AAS [70], [71]. XGBoost Regressor [72] has been used in AAS [71] and SAEAs [73]. Descriptions of these ML models are provided in the supplementary material.

However, when applying these standard ML models to algorithm runtime data, a fundamental limitation emerges. These algorithms, particularly stochastic and heuristic ones, can run for unexpectedly long times on certain configurations or problem instances, due to inherent randomness or auxiliary techniques such as restarts. When testing multiple algorithm configurations across numerous problem instances, the algorithm runtime typically follows a heavy-tailed distribution. In practice, after a certain time, it becomes clear that some algorithm configurations perform worse than others. To save computational resources, a cut-off time is imposed, terminating any process that exceeds this limit. Algorithms terminated before completion are right-censored, with the censored runtime serving as a lower bound for the actual runtime.

This censoring mechanism creates a critical challenge for standard ML approaches. The surrogate model is trained on data containing these censored runs, but the aforementioned standard ML models cannot directly handle censored observations, lacking the theoretical foundation to do so properly. Since the censored runtimes represent lower bounds for actual runtimes, the distribution becomes skewed toward lower values, potentially causing models trained on such data to produce overoptimistic predictions.

This limitation establishes the third desirable property of surrogate models: proper handling of censored runs in the training dataset. Survival analysis provides a well-established class of methods specifically designed to handle censored data. In the experiments presented in Section IV, we consider six survival-based models described in Section II-C.

D. Surrogate model management strategies for ACPP

Surrogate model replaces part of the evaluations of the algorithm configurations on the training problem instances. We propose five evaluation-level and iteration-level surrogate

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

model management strategies. Evaluation-level strategies determine whether to substitute a single evaluation with a surrogate model, while iteration-level strategies determine whether to replace all evaluations within an iteration across all training problem instances.

Evaluation-level strategies:

- **X%-random:** fixed X% probability of employing a surrogate model for each evaluation;
- **X%-shortest:** surrogate model replaces all evaluations; only the X% of configurations with the shortest predicted runtimes undergo re-evaluation with true solvers;
- **cut-off-X%-predicted:** true solver re-evaluation cut-off time is established at X% of the predicted runtime.

Iteration-level strategies:

- **X-Y-iteration:** surrogate model is employed for X consecutive iterations, followed by Y iterations using true solvers;
- **iteration-better-than-incumbent:** surrogate model replaces all evaluations within an iteration; true solvers are utilized only when the new portfolio configuration exceeds the incumbent performance.

Detailed pseudo-code for surrogate model management strategies is provided in the supplementary material in Algorithms 1–5.

IV. Experiments

This section presents the experiments conducted to evaluate the proposed methodology. Section IV-A compares candidate surrogate models, while Section IV-B evaluates model management strategies. All code and data required to reproduce the experiments are released in Zenodo repository [74].

A. Comparison of candidate surrogate models

The initial experimental phase consisted of training and evaluating the models introduced in Section III-C on algorithm runtime data to identify the most suitable surrogate modeling candidate. Since portfolio construction is computationally intensive and time-consuming, a direct comparison through repeated portfolio training with different surrogate models would be prohibitively expensive. Therefore, we first generate algorithm runtime data as described below in Section IV-A1. Then we propose a model comparison framework involving repeated training and testing on data sampled from the algorithm runtime dataset, described in Section IV-A2. The results and discussion are presented in Section IV-A3.

1) Generation of the algorithm runtime dataset: To avoid resource-intensive portfolio construction, we compare candidate models using an algorithm runtime dataset. This dataset contains solver parameters, problem features, and solution times analogous to data generated during portfolio construction. We executed 1000 randomly selected LKH [35], [36] configurations on 100 generated TSP instances, yielding 100,000 evaluations. We measured CPU time of single LKH runs. LKH configurations were characterized by 23 parameter values, while TSP instances were described using 64 features from the `tspmeta` package [75]. Instance sizes were selected randomly and ranged from 50 to 600 cities. Five TSP generators contributed equally: the *clustered* generator from `netgen` [76], and the *compression*, *expansion*, *explosion*, and *grid* generators from `tspgen` [77]. A 300-second cut-off time was applied to limit censored observations, resulting in approximately 20% of runs being terminated at this threshold.

2) Model comparison setup: The models from Section III-C were divided into two categories: six standard ML models (Ridge Regression, Polynomial Regression, Support Vector Regression, Gaussian Process Regression with RBF kernel, Random Forest, and XGBoost) and six survival-based models (Cox proportional hazards model, Gradient Boosting with Cox's partial likelihood loss, Random Survival Forest, XGBoost with Accelerated Failure Time loss, a neural network with Tobit loss function, and Schmee & Hahn imputation with Quantile Regression Forest).

Conventional ML models were trained in two setups: ignoring censoring information (i.e., treating censored data as uncensored and using their recorded values directly) or removing censored observations from the training dataset. Hyperparameters for each model were optimized using SMAC3 [28] with 500 trials, employing 3-fold cross-validation RMSE as the metric on a fixed subset of 10 instances and 90 solver configurations (900 evaluations). This dataset remained constant across all models during hyperparameter optimization. Details regarding hyperparameter search spaces are provided in the supplementary material.

ML models' performance data was collected using 20 independent repetitions. For each repetition, a different random seed was used to draw subsets from the precomputed runtime dataset of 100,000 evaluations (see Figure 2 in the supplementary material for details). To mimic the progressive data accumulation during portfolio construction, we varied the dataset size according to the number of solver configurations $C \in \{5, 10, 15, 20, 30, 50, 70, 100, 150, 200, 300, 500\}$. For each C and each seed, a random subset of 10 instances and C solver configurations was drawn (without replacement) from the precomputed dataset. Models underwent 5-fold cross-validation with recording of the average RMSE on testing folds and the CPU time required for training. We report training time measurements in the supplementary material.

Due to the log-normal-like runtime distribution, the runtimes were logarithmically transformed before the model training and RMSE calculation. For each configuration count, the best-performing model was identified using the Wilcoxon signed-rank test [78], with the significance level set to 5%, to determine statistical significance of performance differences.

To evaluate candidate models' ability to handle censored training data, we created two scenarios.

a) Constant 1 second cut-off time: The algorithm runtime dataset (Section IV-A1) was generated with a 300-second cut-off. In this scenario, we artificially set all runtimes exceeding 1 second to exactly 1 second and marked them as censored, resulting in approximately 67% censored evaluations. This modification was applied to training data only. The model predictions were compared with the true run-time values (including those exceeding 1 second). This scenario tests models' ability to predict values beyond a constant cut-

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

TABLE I

RMSE RESULTS FOR ALGORITHM RUNTIME PREDICTION ACROSS TWO CUT-OFF SCENARIOS. VALUES REPRESENT RMSE CALCULATED ON LOGARITHMIZED ACTUAL AND PREDICTED ALGORITHM RUNTIMES, AVERAGED OVER 5-FOLD CROSS-VALIDATION TESTING FOLDS AND 20 RANDOM SEEDS, FOR DATASETS OF VARYING SIZES. IN EACH COLUMN (FOR EACH DATASET SIZE), THE BEST MODEL IS HIGHLIGHTED IN YELLOW. BOLDFACE INDICATES PERFORMANCE NOT SIGNIFICANTLY DIFFERENT FROM THE BEST METHOD IN EACH COLUMN. (I) DENOTES MODELS TRAINED IGNORING CENSORING INFORMATION; (R) DENOTES MODELS WITH CENSORED OBSERVATIONS REMOVED.

Constant 1-second cut-off time												
Model	5	10	15	20	30	50	70	100	150	200	300	500
Ridge Regression (i)	3.27	3.55	3.36	3.30	4.06	3.54	3.39	3.44	3.47	3.41	3.49	3.40
Polynomial Regression (i)	3.29	3.55	3.45	3.41	3.63	3.64	3.52	3.57	3.58	3.50	3.53	3.41
Support Vector Regression (i)	3.26	3.50	3.36	3.30	3.50	3.51	3.39	3.43	3.45	3.38	3.45	3.36
Gaussian Process Regression (i)	3.23	3.44	3.33	3.29	3.50	3.52	3.40	3.47	3.50	3.44	3.51	3.42
Random Forest (i)	3.40	3.62	3.52	3.38	3.61	3.60	3.48	3.50	3.51	3.43	3.49	3.39
XGBoost (i)	3.37	3.63	3.52	3.39	3.60	3.61	3.49	3.51	3.51	3.43	3.48	3.36
Ridge Regression (r)	3.79	3.83	3.56	3.56	3.79	3.93	3.63	3.64	3.68	3.58	3.70	3.61
Polynomial Regression (r)	4.38	4.56	4.28	4.00	4.33	4.28	4.08	4.05	3.98	3.81	3.75	3.44
Support Vector Regression (r)	3.83	3.84	3.73	3.64	3.87	3.86	3.90	3.94	4.14	4.15	4.45	4.73
Gaussian Process Regression (r)	3.25	3.44	3.33	3.29	3.50	3.53	3.40	3.47	3.51	3.47	3.56	3.53
Random Forest (r)	3.98	4.30	4.06	3.94	4.27	4.25	4.09	4.16	4.16	4.09	4.14	4.03
XGBoost (r)	4.07	4.26	4.07	3.94	4.25	4.20	4.05	4.11	4.07	3.97	3.98	3.78
Cox Proportional Hazards	**2.81**	**2.69**	**2.67**	**2.66**	**2.65**	2.63	2.60	2.56	2.58	2.52	2.55	2.59
Random Survival Forest	**2.84**	**2.81**	2.81	**2.76**	2.74	2.69	2.70	2.64	2.59	2.55	2.52	2.49
Gradient Boosting Cox	**2.83**	**2.75**	**2.78**	**2.68**	**2.59**	**2.53**	2.48	2.44	2.40	2.34	2.35	2.31
XGBoost Accelerated Failure Time	**3.21**	3.01	**2.90**	**2.69**	**2.54**	**2.45**	**2.34**	**2.22**	**2.08**	**2.04**	**1.90**	**1.79**
Schmee & Hahn Quantile Regression Forest	**3.12**	3.30	3.18	3.04	3.17	3.22	3.06	3.12	3.14	3.11	3.14	3.07
Neural Network Tobit	**3.01**	3.00	**2.74**	**2.73**	2.79	**2.62**	2.65	2.49	2.45	2.39	2.36	2.26

Cut-offs scaled according to problem size												
Model	5	10	15	20	30	50	70	100	150	200	300	500
Ridge Regression (i)	**2.94**	3.13	2.92	2.95	3.86	2.94	2.95	2.93	2.96	2.91	2.95	2.95
Polynomial Regression (i)	**2.96**	3.11	3.04	3.06	3.17	3.10	3.12	3.11	3.11	3.04	3.01	2.96
Support Vector Regression (i)	**3.03**	3.02	2.91	2.92	2.97	2.90	2.93	2.91	2.93	2.89	2.92	2.92
Gaussian Process Regression (i)	**2.95**	3.10	2.94	2.93	3.03	2.94	2.95	2.93	2.94	2.88	2.91	2.89
Random Forest (i)	**2.98**	3.15	3.07	2.97	3.09	3.00	3.04	2.99	3.00	2.95	2.97	2.98
XGBoost (i)	**2.99**	3.11	3.05	2.97	3.10	2.99	3.01	2.96	2.95	2.90	2.90	2.88
Ridge Regression (r)	3.59	3.58	3.31	3.38	3.50	3.61	3.42	3.39	3.37	3.32	3.36	3.36
Polynomial Regression (r)	3.95	3.88	3.66	3.58	3.77	3.67	3.67	3.64	3.58	3.47	3.40	3.21
Support Vector Regression (r)	3.59	3.61	3.35	3.40	3.51	3.51	3.43	3.36	3.34	3.27	3.30	3.30
Gaussian Process Regression (r)	**3.22**	3.44	3.32	3.28	3.49	3.50	3.38	3.45	3.48	3.43	3.51	3.46
Random Forest (r)	3.84	3.86	3.63	3.59	3.82	3.70	3.71	3.73	3.72	3.67	3.68	3.68
XGBoost (r)	3.47	3.66	3.49	3.51	3.70	3.54	3.55	3.49	3.38	3.30	3.24	3.11
Cox Proportional Hazards	**2.82**	**2.67**	**2.57**	**2.56**	**2.50**	**2.45**	**2.42**	2.34	2.32	2.26	2.28	2.27
Random Survival Forest	**2.92**	2.93	2.87	2.81	2.80	2.77	2.75	2.70	2.65	2.62	2.59	2.55
Gradient Boosting Cox	3.10	**2.91**	2.89	2.73	**2.50**	**2.39**	2.38	2.27	2.20	2.19	2.16	2.20
XGBoost Accelerated Failure Time	3.30	**3.00**	2.87	**2.69**	**2.47**	**2.35**	**2.30**	**2.17**	**2.02**	**1.97**	**1.84**	**1.72**
Schmee & Hahn Quantile Regression Forest	**2.88**	**2.79**	2.78	**2.59**	**2.45**	**2.39**	2.43	2.28	2.23	2.13	2.09	2.03
Neural Network Tobit	**2.95**	2.90	**2.63**	**2.62**	2.68	**2.54**	2.43	**2.20**	**2.08**	**2.04**	1.96	1.89

off when training data contains no observations exceeding this censoring threshold.

b) Cut-offs scaled according to the problem size: The second scenario established cut-off times based on problem size, with a reference cut-off of 10 seconds for 600-city TSP instances. For instances with n cities, the cut-off was calculated as $10 \cdot \left(\frac{n}{600}\right)^{2.2}$, reflecting the $O(n^{2.2})$ complexity of the LKH solver. This approach resulted in approximately 60% censored evaluations.

3) Model comparison results: Table I presents numerical results for the two above-discussed cut-off scenarios. RMSE values were computed on log-transformed actual and predicted algorithm runtimes, averaged over 5-fold cross-validation and 20 random seeds across datasets of varying sizes.

Standard ML models trained after removing censored observations exhibit the poorest performance. Models retaining all observations while ignoring censoring performed marginally better but were comparable to survival-based models only for datasets with fewer than 20 solver configurations.

The Cox proportional hazards model was either the best-performing or statistically equivalent to the best model for datasets with 30 or fewer solver configurations across all cut-off strategies. For datasets exceeding 20 solver configurations, XGBoost with Accelerated Failure Time loss consistently achieved optimal performance or was not significantly different from the best model. Other survival-based models occasionally matched top performance for specific dataset sizes but lacked consistent superiority across variable conditions.

Figures 1 and 2 compare actual versus predicted runtimes for the constant 1-second cut-off and problem-size-scaled cut-off, respectively. For the sake of clarity, standard ML models trained on data with censored observations removed

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

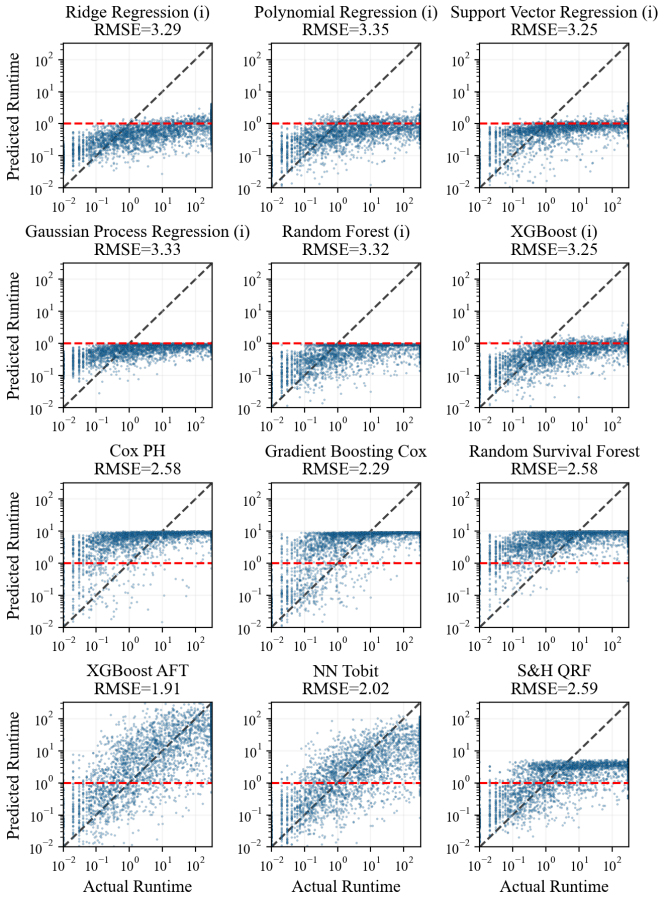


Fig. 1. Performance comparison of surrogate models on algorithm runtime data from 10 TSP instances with 300 LKH configurations. Training data uses constant 1-second cut-offs for all observations (red dashed line). Scatterplots display actual runtime (X-axis) versus predicted runtime (Y-axis) for each model across 5-fold cross-validation testing folds. RMSE values are computed from logarithmized actual and predicted runtimes. The black dashed line indicates perfect prediction.

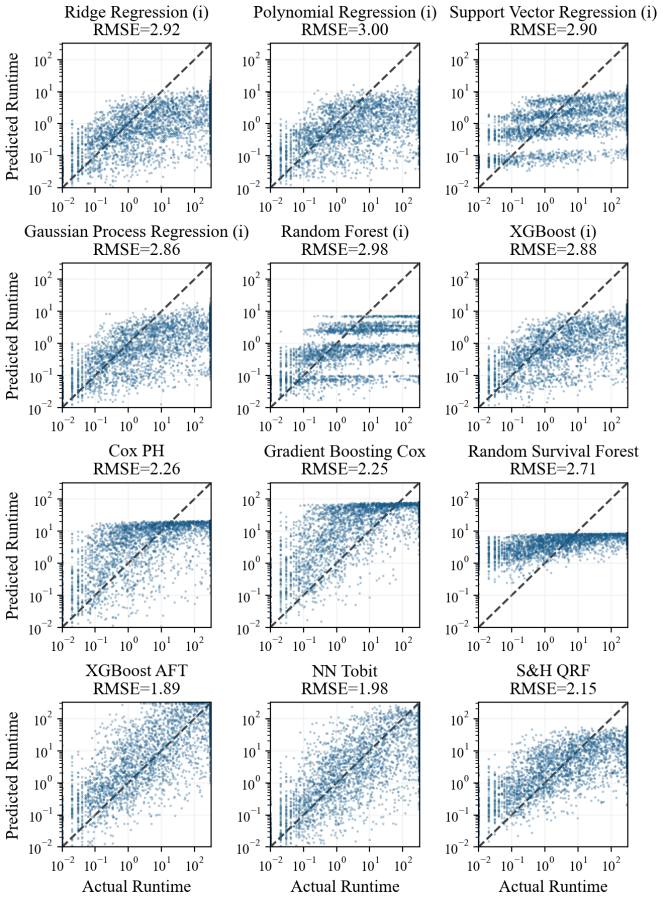


Fig. 2. Performance comparison of surrogate models on algorithm runtime data from 10 TSP instances with 300 LKH configurations. Training data uses problem size-scaled cut-offs: 0.04s for 50 cities and 10.0s for 600 cities. Scatterplots display actual runtime (X-axis) versus predicted runtime (Y-axis) for each model across 5-fold cross-validation testing folds. RMSE values are computed from logarithmized actual and predicted runtimes. The black dashed line indicates perfect prediction.

are omitted. Figure 3 shows boxplots of mean and variance for survival-based models for the problem-size-scaled cut-off scenario across increasing dataset sizes.

Cox proportional hazards, Gradient Boosting with Cox's partial likelihood, and Random Survival Forests cannot predict event times exceeding the largest observed runtime in the training data, as their estimated survival functions are defined only up to this limit [44]. In this scenario, hyperparameter optimization selected the PAR10 risk function, which maps censored runs to $10 \cdot C$, resulting in predicted runtimes capped at 10 seconds, as presented in Figure 1.

Overall, survival-based models demonstrate superior predictive performance across all scenarios. This advantage is particularly evident in Figure 1, where standard ML models fail to predict runtimes beyond the imposed 1-second cut-off.

B. Comparison of model management strategies in PAP construction

This section presents the second set of experiments, evaluating surrogate model management approaches detailed in Section III-D with respect to portfolio construction time reduction and portfolio performance on unseen test instances. The experimental configuration is outlined in Section IV-B1, the results and their analysis are presented in Section IV-B2, and section IV-B3 provides conclusions. Section X of the supplementary material presents additional experiments demonstrating the applicability of the proposed methods across other benchmarks, larger portfolios, alternative solvers, different problem types, and AAC procedures beyond SMAC3.

1) Model management strategies comparison setup: The *iteration-better-than-incumbent* strategy requires no hyperparameter tuning, while other strategies were tested with the following hyperparameter values:

- **X%-random** with X = 25%, 50%, and 75%;
- **X%-shortest** with X = 25%, 50%, and 75%;
- **cut-off-X%-predicted** with X = 100%, 75%, and 50%;
- **X-Y-iteration** with X-Y = 1-2, 1-1, and 2-1.

The above strategies were compared with a baseline approach without surrogate model implementation.

PARHYDRA was used for ACPP throughout the experiments, with SMAC3 serving as the underlying AAC proce-

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

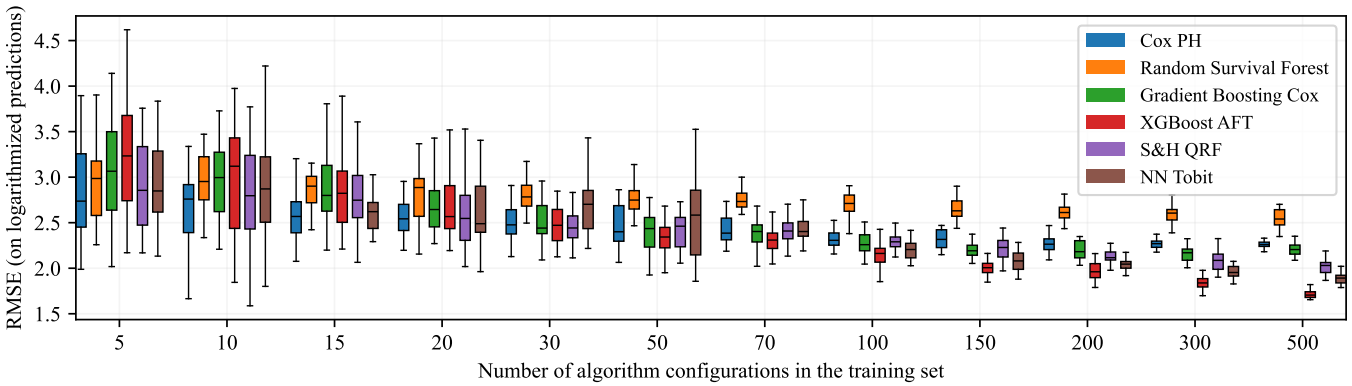


Fig. 3. RMSE performance for problem size-scaled cut-off scenario. Boxplots show RMSE values computed from logarithmized actual and predicted algorithm runtimes, averaged over 5-fold cross-validation, across 20 random seeds. Results represent random subsets of 10 instances with varying numbers of solver configurations $C \in \{5, 10, 15, 20, 30, 50, 70, 100, 150, 200, 300, 500\}$ from the algorithm runtime dataset.

dure. The objective function f for configuration was PAR10, i.e., the solver runtime multiplied by 10 when the solver hit the cut-off. There were two component solvers, each configured for 4 attempts of 25 iterations ($MaxAttempt = 4$ and $MaxIter = 25$). Portfolio training sets contained 25, 100, or 250 problem instances from five TSP generators (*clustered*, *compression*, *expansion*, *explosion*, and *grid*) with equal representation. Test sets contained 250 problem instances from ten generators: the five training generators plus *random*, *implosion*, *cluster*, *rotation*, and *linearprojection*. TSP instances in both the training and test sets ranged from 50 to 600 cities. Training phase cut-offs were scaled according to the formula $10 \cdot \left(\frac{n}{600}\right)^{2.2}$ that reflects the LKH solver time scalability. The test phase cut-off time was set to 100 seconds to avoid censored evaluations.

For strategies other than the baseline, surrogate model training began after 5 iterations and was repeated every 5 iterations thereafter. The Cox proportional hazards model was used when the number of unique solvers in the collected dataset was less than or equal to 20. After a larger dataset was collected, XGBoost with the Accelerated Failure Time loss was used as a surrogate. The portfolio training was repeated 20 times for each training set size and hyperparameter value. Each portfolio solved each test problem instance 5 times, with median times recorded. Total CPU training time, defined as the time required to evaluate all configurations proposed during training, as well as the average time required to solve test instances were recorded. Results from model management strategies were compared against the baseline approach. Ratios of training CPU time and average runtimes on test instances relative to the baseline approach were calculated to assess training time reduction and performance impact on new, unseen problem instances.

We hypothesized that increased surrogate model usage would degrade performance on unseen problem instances compared to the baseline, as surrogate predictions are less accurate than true solver evaluations and may mislead the AAC procedure.

2) Model management strategies comparison results: Table II presents total CPU time for portfolio training with

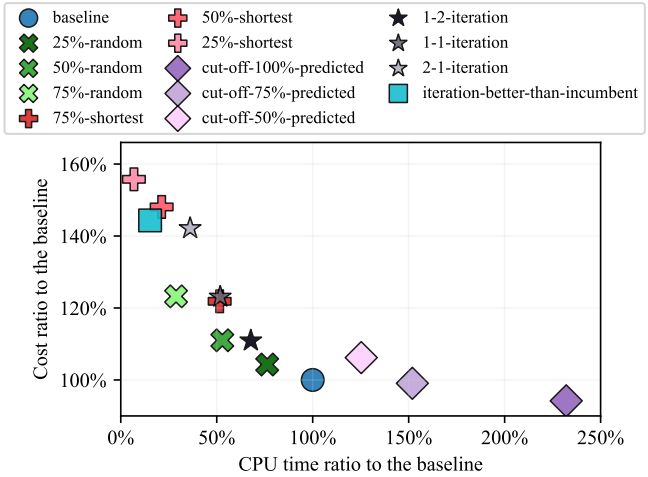


Fig. 4. Ratios of the training CPU time (X-axis) and test instance runtime (Y-axis) relative to the baseline for different portfolio training strategies with the training sample size equal to 25. The ratios are averaged over 20 experiment repetitions.

each surrogate model management strategy (columns 2-7) and average portfolio runtime to find an optimum averaged across all test problem instances (columns 8-13). The feature extraction costs are included in the total CPU portfolio training time. These costs are negligible, since feature extraction is performed only once at the beginning of the portfolio training, as the training instances remain the same over the training process. The statistics for feature extraction time are provided in the supplementary material. The comparison of the portfolio performance between the two separate groups of generators is provided in the supplementary material. Separating the generator groups did not qualitatively change the results and conclusions.

It can be seen from Table II that portfolio training time scales proportionally with training sample size, while sample size has minimal impact on training time ratios relative to the baseline. The ratio columns clearly demonstrate surrogate model effects for the *X%-random* strategy: 25% random usage achieves approximately 25% training time reduction compared

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

TABLE II

TRAINING CPU TIME [H] AND TEST PERFORMANCE [S] FOR PORTFOLIO TRAINING STRATEGIES ACROSS DIFFERENT SAMPLE SIZES (25, 100, 250). TRAINING TIMES SHOW THE TOTAL CPU HOURS REQUIRED, WHILE TEST PERFORMANCE SHOWS MEAN RUNTIME TO FIND THE OPTIMUM ON 250 TSP INSTANCES (50-600 CITIES). PERCENTAGE RATIOS ARE CALCULATED RELATIVE TO THE BASELINE FOR EACH SAMPLE SIZE. RESULTS ARE AVERAGED OVER 20 EXPERIMENTAL REPETITIONS. BOLDFACE INDICATES PERFORMANCE NOT SIGNIFICANTLY DIFFERENT FROM THE BASELINE, AS INDICATED BY THE WILCOXON SIGNED-RANK TEST. COLOR GRADIENTS HIGHLIGHT PERFORMANCE DIFFERENCES WITHIN EACH METRIC GROUP.

	Training						Test					
	CPU time [h]			Ratio to baseline			Mean runtime [s]			Ratio to baseline		
	Training sample size			Training sample size			Training sample size			Training sample size		
Strategy	25	100	250	25	100	250	25	100	250	25	100	250
baseline	3.44	13.18	33.47	100%	100%	100%	**0.170**	**0.152**	**0.151**	100%	100%	100%
25%-random	2.62	9.92	25.23	76%	75%	75%	**0.178**	0.170	0.169	104%	112%	112%
50%-random	1.82	6.81	17.11	53%	52%	51%	**0.189**	0.181	0.176	111%	120%	116%
75%-random	0.99	3.66	9.14	29%	28%	27%	**0.210**	0.203	0.192	123%	134%	127%
75%-shortest	1.77	6.69	15.74	52%	51%	47%	0.208	0.198	0.195	122%	131%	129%
50%-shortest	0.73	2.54	6.17	21%	19%	18%	0.252	0.220	0.212	148%	145%	140%
25%-shortest	0.24	0.83	1.97	7%	6%	6%	0.265	0.260	0.242	156%	171%	160%
cut-off-100%-predicted	7.98	31.01	76.10	232%	235%	227%	**0.161**	**0.159**	**0.155**	94%	105%	103%
cut-off-75%-predicted	5.22	21.47	53.83	152%	163%	161%	**0.169**	0.173	**0.160**	99%	114%	106%
cut-off-50%-predicted	4.31	16.58	41.50	125%	126%	124%	**0.181**	0.177	0.175	106%	117%	116%
1-2-iteration	2.33	8.96	22.97	68%	68%	69%	**0.189**	0.179	0.178	111%	118%	118%
1-1-iteration	1.78	6.73	17.12	52%	51%	51%	0.210	0.188	0.186	123%	124%	123%
2-1-iteration	1.24	4.70	12.06	36%	36%	36%	0.242	0.217	0.211	142%	143%	139%
iteration-better-than-incumbent	0.53	2.12	5.58	15%	16%	17%	0.246	0.210	0.207	144%	138%	137%

to the baseline. For *50%-random* and *75%-random*, time savings are slightly below 50% and 75%, respectively, likely due to reduced portfolio performance during training. The *X%-shortest* strategy achieves greater training time reduction than *X%-random* with equivalent surrogate usage.

The results on test problem instances presented in Table II confirm our intuition that surrogate model usage decreases performance. The larger the portfolio training set size, the better the results on the testing set.

The *cut-off-X%-predicted* strategy requires more training time than the baseline with no performance improvement, indicating its limited utility. The reason is that all problem instances are evaluated with true solvers while the surrogate model predicts the required solving time and sets the cut-off accordingly. These predicted cut-offs are often higher than those used in the baseline, where the cut-off is determined by the formula $10 \cdot \left(\frac{n}{600}\right)^{2.2}$, leading to longer algorithm runs during portfolio construction.

We can compare strategies by examining training time reduction and performance degradation on unseen test instances. Figure 4 plots both metrics, i.e., CPU time ratio (X-axis) and test set cost (Y-axis), both relative to the baseline.

Generally, the *X-Y-iteration* strategy underperforms *X%-random*, yielding greater performance loss for similar training savings. For instance, *1-1-iteration* and *50%-random* both achieve 50% training time reduction, but *1-1-iteration* underperforms baseline by 23%-39% on testing, while *50%-random* shows only 11%-20% degradation.

However, the results in Table II and Figure 4 do not permit a direct comparison of strategies that differ in both training time and test performance. For example, the baseline and *25%-random* cannot be compared directly, as the results do not allow us to infer what performance the baseline would achieve if trained for the same amount of time as *25%-random*, nor how much training time *25%-random* would require to attain the same test performance as the baseline.

To address this limitation, we additionally present the portfolio construction experiments using a fixed CPU training time budget instead of a fixed iteration count. The results for portfolios trained on 25 problem instances under three training budgets (1h, 3h, and 5h) are presented in Table III. With a fixed training time budget, test performance can be directly compared across strategies. Generally, using surrogates is beneficial: for all training budgets, the *X%-random* strategies achieve better test performance than the baseline. The same holds for the *X%-shortest* strategies (except for two cases), the *X-Y-iteration* strategies (except for one case), and the *iteration-better-than-incumbent* strategy (for two out of three budgets). Across all three training budgets, the best performance is achieved by the *75%-random* strategy, which improves test performance by 2%–14% over the baseline. These differences, though apparent in the mean runtimes, are not detected by the Wilcoxon signed-rank test.

3) Conclusions from the comparison of surrogate model management strategies: The proposed surrogate model management strategies were compared in terms of portfolio construction time reduction and performance impact on new, unseen test problem instances. Generally, surrogate models accelerate portfolio construction, but at the cost of reduced test performance (when the number of training iterations is fixed). The magnitude of this impact, however, depends on the specific strategy employed. For example, the *50%-random*, *75%-shortest*, and *1-1-iteration* strategies all provide about 50% time reduction, but the test performance degradation is 11%–20% for the first, 22%–31% for the second, and 23%–24% for the last strategy. Note, however, that with an equal training time budget, the proposed methods seem to improve test performance compared to the baseline, although the differences are not statistically significant.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

TABLE III
TEST PERFORMANCE [S] FOR PORTFOLIO TRAINING STRATEGIES ACROSS DIFFERENT TRAINING BUDGETS (1H, 3H, 5H). PORTFOLIO TRAINING WAS CONDUCTED USING 25 PROBLEM INSTANCES. TEST PERFORMANCE IS MEASURED AS THE MEAN RUNTIME TO FIND THE OPTIMUM ON 250 TSP INSTANCES (50-600 CITIES). RESULTS ARE AVERAGED OVER 20 EXPERIMENTAL REPETITIONS. BOLDFACE INDICATES PERFORMANCE THAT IS NOT SIGNIFICANTLY DIFFERENT FROM THE BEST IN EACH COLUMN, ACCORDING TO THE WILCOXON SIGNED-RANK TEST. COLOR GRADIENTS HIGHLIGHT PERFORMANCE DIFFERENCES WITHIN EACH COLUMN.

Strategy	Mean runtime [s] Training budget in CPU hours 1h	3h	5h
baseline	**0.2494**	**0.1747**	**0.1628**
25%-random	0.2365	**0.1738**	**0.1626**
50%-random	**0.2228**	**0.1724**	**0.1602**
75%-random	**0.2137**	**0.1715**	**0.1592**
75%-shortest	0.2478	**0.1756**	**0.1621**
50%-shortest	**0.2328**	**0.1736**	**0.1604**
25%-shortest	**0.2227**	**0.1752**	**0.1593**
cut-off-100%-predicted	0.2707	0.2124	0.1898
cut-off-75%-predicted	0.2625	0.2027	0.1769
cut-off-50%-predicted	0.2529	0.1966	**0.1783**
1-2-iteration	0.2451	**0.1759**	**0.1620**
1-1-iteration	**0.2393**	**0.1734**	**0.1605**
2-1-iteration	0.2412	**0.1728**	**0.1616**
iteration-better-than-incumbent	0.2563	**0.1727**	**0.1601**

V. CONCLUSIONS

This paper addresses the previously unexplored problem of accelerating PAP construction by replacing part of the costly algorithm evaluations during portfolio training with surrogate model predictions. We begin with establishing three essential properties for ML models used as surrogates in portfolio construction: predictive quality, low data requirements, and the ability to handle censored runtimes. The third property is particularly important because cut-off times terminate unsuccessful algorithm configurations early while introducing bias toward lower runtime predictions in the model training data.

We systematically evaluated twelve candidate surrogate models on LKH solver runtime data for TSP instances: six conventional ML models and six survival analysis approaches. The Cox proportional hazards model and XGBoost with Accelerated Failure Time loss function emerged as superior surrogates, with the former preferred when runtime data is scarce, while the latter one when sufficient data is available.

We developed and compared five surrogate model management strategies that define when to replace actual evaluations with surrogate predictions during portfolio construction. Under a fixed iteration budget, these strategies achieved substantial training time reductions (e.g., approximately 50% for the *50%-random* strategy) with moderate test performance degradation (11%–20% for the same strategy). Experiments with a fixed CPU training time budget confirmed that surrogate-based strategies typically outperform the baseline trained for the same duration. The *75%-random* strategy yielded the highest test performance, improving upon the baseline by 2%–14% across CPU time budgets of 1, 3, and 5 hours.

In future work, we plan to investigate alternative surrogate model management strategies, particularly those that incorporate prediction confidence measures to make more informed decisions about when to trust surrogate predictions versus actual algorithm evaluations.

REFERENCES

[1] D. Wolpert and W. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, Apr. 1997.

[2] H. H. Hoos, "Automated algorithm configuration and parameter tuning," in *Autonomous Search*, Y. Hamadi, E. Monfroy, and F. Saubion, Eds. Springer Berlin Heidelberg, 2012, ch. 3, pp. 37–71.

[3] E. Schede, J. Brandt, A. Tornede, M. Wever, V. Bengs, E. Hüllermeier, and K. Tierney, "A Survey of Methods for Automated Algorithm Configuration," *Journal of Artificial Intelligence Research*, vol. 75, pp. 425–487, Oct. 2022.

[4] C. Huang, Y. Li, and X. Yao, "A Survey of Automatic Parameter Tuning Methods for Metaheuristics," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 2, pp. 201–216, 2020.

[5] F. Hutter, Holger. H. Hoos, K. Leyton-Brown, and T. Stuetzle, "ParamILS: An Automatic Algorithm Configuration Framework," *Journal of Artificial Intelligence Research*, vol. 36, pp. 267–306, Oct. 2009.

[6] C. Ansótegui, M. Sellmann, and K. Tierney, "A Gender-Based Genetic Algorithm for the Automatic Configuration of Algorithms," in *Principles and Practice of Constraint Programming - CP 2009*, I. P. Gent, Ed. Berlin, Heidelberg: Springer, 2009, pp. 142–157.

[7] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, Jan. 2016.

[8] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential Model-Based Optimization for General Algorithm Configuration," in *Learning and Intelligent Optimization*, C. A. C. Coello, Ed. Berlin, Heidelberg: Springer, 2011, pp. 507–523.

[9] K. Asanovic, R. Bodik, J. Demmel, T. Keaveny, K. Keutzer, J. Kubiatowicz, N. Morgan, D. Patterson, K. Sen, J. Wawrzynek, D. Wessel, and K. Yelick, "A view of the parallel computing landscape," *Communications of the ACM*, vol. 52, no. 10, pp. 56–67, Oct. 2009.

[10] Y. Hamadi and C. Wintersteiger, "Seven challenges in parallel SAT solving," *Ai Magazine*, vol. 34, no. 2, pp. 99–99, 2013.

[11] B. A. Huberman, R. M. Lukose, and T. Hogg, "An Economics Approach to Hard Computational Problems," *Science*, vol. 275, no. 5296, pp. 51–54, Jan. 1997.

[12] C. P. Gomes and B. Selman, "Algorithm portfolios," *Artificial Intelligence*, vol. 126, no. 1, pp. 43–62, Feb. 2001.

[13] R. Akay, A. Basturk, A. Kalinli, and X. Yao, "Parallel population-based algorithm portfolios: An empirical study," *Neurocomputing*, vol. 247, pp. 115–125, 2017.

[14] K. Tang, F. Peng, G. Chen, and X. Yao, "Population-based Algorithm Portfolios with automated constituent algorithms selection," *Information Sciences*, vol. 279, pp. 94–104, 2014.

[15] M. Lindauer, H. Hoos, K. Leyton-Brown, and T. Schaub, "Automatic construction of parallel portfolios via algorithm configuration," *Artificial Intelligence*, vol. 244, pp. 272–290, Mar. 2017.

[16] L. Xu, H. Hoos, and K. Leyton-Brown, "Hydra: Automatically configuring algorithms for portfolio-based selection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, 2010, pp. 210–216.

[17] S. Kadioglu, Y. Malitsky, M. Sellmann, and K. Tierney, "ISAC–instance-specific algorithm configuration," in *ECAI 2010*. IOS Press, 2010, pp. 751–756.

[18] S. Liu, K. Tang, and X. Yao, "Automatic Construction of Parallel Portfolios via Explicit Instance Grouping," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 1560–1567, Jul. 2019.

[19] ——, "Generative Adversarial Construction of Parallel Portfolios," *IEEE Transactions on Cybernetics*, vol. PP, pp. 1–12, Apr. 2020.

[20] K. Tang, S. Liu, P. Yang, and X. Yao, "Few-Shots Parallel Algorithm Portfolio Construction via Co-Evolution," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 3, pp. 595–607, Jun. 2021.

[21] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, Jun. 2011.

[22] H. Tong, C. Huang, L. L. Minku, and X. Yao, "Surrogate models in evolutionary single-objective optimization: A new taxonomy and experimental study," *Information Sciences*, vol. 562, pp. 414–437, Jul. 2021.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

[23] M. Okulewicz, M. Zaborski, and J. Mańdziuk, "Self-Adapting Particle Swarm Optimization for continuous black box optimization," *Applied Soft Computing*, vol. 131, p. 109722, Dec. 2022.

[24] J. R. Rice, "The algorithm selection problem," in *Advances in Computers*. Elsevier, 1976, vol. 15, pp. 65–118.

[25] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated Algorithm Selection: Survey and Perspectives," *Evolutionary Computation*, vol. 27, no. 1, pp. 3–45, Mar. 2019.

[26] R. Tanabe, "Benchmarking Feature-Based Algorithm Selection Systems for Black-Box Numerical Optimization," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 6, pp. 1321–1335, Dec. 2022.

[27] L. Meunier, H. Rakotoarison, P. K. Wong, B. Roziere, J. Rapin, O. Teytaud, A. Moreau, and C. Doerr, "Black-Box Optimization Revisited: Improving Algorithm Selection Wizards Through Massive Benchmarking," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 3, pp. 490–500, Jun. 2022.

[28] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A Versatile Bayesian Optimization Package for Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.

[29] A. Tornede, M. Wever, S. Werner, F. Mohr, and E. Hüllermeier, "Run2Survive: A Decision-theoretic Approach to Algorithm Selection based on Survival Analysis," in *Proceedings of The 12th Asian Conference on Machine Learning*. PMLR, Sep. 2020, pp. 737–752.

[30] J. Hanselle, A. Tornede, M. Wever, and E. Hüllermeier, "Algorithm selection as superset learning: Constructing algorithm selectors from imprecise performance data," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2021, pp. 152–163.

[31] F. Hutter, L. Xu, H. H. Hoos, and K. Leyton-Brown, "Algorithm runtime prediction: Methods & evaluation," *Artificial Intelligence*, vol. 206, pp. 79–111, Jan. 2014.

[32] K. Eggensperger, M. Lindauer, and F. Hutter, "Neural Networks for Predicting Algorithm Runtime Distributions," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, Jul. 2018, pp. 1442–1448.

[33] K. Eggensperger, M. Lindauer, H. H. Hoos, F. Hutter, and K. Leyton-Brown, "Efficient benchmarking of algorithm configurators via model-based surrogates," *Machine Learning*, vol. 107, no. 1, pp. 15–41, Jan. 2018.

[34] G. Zakrzewski, X. Yao, and J. Mańdziuk, "Accelerating parallel algorithm portfolio construction," *Procedia Computer Science*, vol. 270, pp. 1159–1168, 2025, 29th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2025).

[35] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *European Journal of Operational Research*, vol. 126, no. 1, pp. 106–130, Oct. 2000.

[36] ——, "General k-opt submoves for the Lin–Kernighan TSP heuristic," *Mathematical Programming Computation*, vol. 1, no. 2-3, pp. 119–163, Oct. 2009.

[37] G. Reinelt, "TSPLIB—A Traveling Salesman Problem Library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.

[38] K. Tang and X. Yao, "Learn to optimize—a brief overview," *National Science Review*, vol. 11, no. 8, p. nwae132, Aug. 2024.

[39] K. Eggensperger, M. Lindauer, and F. Hutter, "Pitfalls and best practices in algorithm configuration," *Journal of Artificial Intelligence Research*, vol. 64, pp. 861–893, 2019.

[40] A. Żychowski, X. Yao, and J. Mańdziuk, "Diversity-driven cooperating portfolio of metaheuristic algorithms," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2025)*, 2025, pp. 863–871.

[41] P. A. Kowalski, S. Kucharczyk, and J. Mańdziuk, "Constrained hybrid metaheuristic algorithm for probabilistic neural networks learning," *Information Sciences*, vol. 713, p. 122185, 2025.

[42] A. Żychowski, X. Yao, and J. Mańdziuk, "Adaptive metaheuristic selection in island-based optimization," *Procedia Computer Science*, vol. 270, pp. 1119–1128, 2025, 29th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2025).

[43] P. Kerschke, L. Kotthoff, J. Bossek, H. H. Hoos, and H. Trautmann, "Leveraging TSP solver complementarity through machine learning," *Evolutionary computation*, vol. 26, no. 4, pp. 597–620, 2018.

[44] D. G. Kleinbaum and M. Klein, *Survival Analysis*, ser. Statistics for Biology and Health. New York, NY: Springer New York, 2012.

[45] D. R. Cox, "Regression Models and Life-Tables," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 34, no. 2, pp. 187–202, Jan. 1972.

[46] N. Breslow, "Covariance analysis of censored survival data," *Biometrics*, pp. 89–99, 1974.

[47] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of statistics*, pp. 1189–1232, 2001.

[48] ——, "Stochastic gradient boosting," *Computational statistics & data analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[49] G. Ridgeway, "The state of boosting," *Computing science and statistics*, pp. 172–181, 1999.

[50] H. Ishwaran, U. B. Kogalur, E. H. Blackstone, and M. S. Lauer, "Random survival forests," *The Annals of Applied Statistics*, vol. 2, no. 3, pp. 841–860, Sep. 2008.

[51] A. Barnwal, H. Cho, and T. Hocking, "Survival Regression with Accelerated Failure Time Model in XGBoost," *Journal of Computational and Graphical Statistics*, vol. 31, no. 4, pp. 1292–1302, Oct. 2022.

[52] K. Eggensperger, K. Haase, P. Müller, M. Lindauer, and F. Hutter, "Neural Model-based Optimization with Right-Censored Observations," Sep. 2020.

[53] J. Schmee and G. J. Hahn, "A Simple Method for Regression Analysis With Censored Data," *Technometrics*, vol. 21, no. 4, pp. 417–432, Nov. 1979.

[54] F. Hutter, H. Hoos, and K. Leyton-Brown, "Bayesian Optimization With Censored Response Data," Oct. 2013.

[55] N. Meinshausen and G. Ridgeway, "Quantile regression forests," *Journal of machine learning research*, vol. 7, no. 6, 2006.

[56] M. Gagliolo and C. Legrand, "Algorithm Survival Analysis," in *Experimental Methods for the Analysis of Optimization Algorithms*, T. Bartz-Beielstein, M. Chiarandini, L. Paquete, and M. Preuss, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 161–184.

[57] B. Bischl, P. Kerschke, L. Kotthoff, M. Lindauer, Y. Malitsky, A. Fréchette, H. Hoos, F. Hutter, K. Leyton-Brown, and K. Tierney, "Aslib: A benchmark library for algorithm selection," *Artificial Intelligence*, vol. 237, pp. 41–58, 2016.

[58] V. Margraf, T. Koerner, A. Tornede, and M. Wever, "Runandschedule2survive: Algorithm scheduling based on run2survive," *ACM Transactions on Evolutionary Learning*, 2025.

[59] M. de Souza, M. Ritt, and M. López-Ibáñez, "Capping methods for the automatic configuration of optimization algorithms," *Computers & Operations Research*, vol. 139, p. 105615, Mar. 2022.

[60] A. Tornede, V. Bengs, and E. Hüllermeier, "Machine Learning for Online Algorithm Selection under Censored Feedback," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 9, pp. 10 370–10 380, Jun. 2022.

[61] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, Feb. 1970.

[62] F. Hutter, Y. Hamadi, H. H. Hoos, and K. Leyton-Brown, "Performance Prediction and Automated Tuning of Randomized and Parametric Algorithms," in *Principles and Practice of Constraint Programming - CP 2006*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, and F. Benhamou, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, vol. 4204, pp. 213–228.

[63] L. Xu, H. H. Hoos, and K. Leyton-Brown, "Hierarchical Hardness Models for SAT," in *Principles and Practice of Constraint Programming – CP 2007*, C. Bessière, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, vol. 4741, pp. 696–711.

[64] T. Hastie, R. Tibshirani, and J. Friedman, "The elements of statistical learning," 2009.

[65] A. Díaz-Manríquez, G. Toscano, and C. A. Coello Coello, "Comparison of metamodeling techniques in evolutionary algorithms," *Soft Computing*, vol. 21, no. 19, pp. 5647–5663, Oct. 2017.

[66] H. Drucker, C. J. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," *Advances in neural information processing systems*, vol. 9, 1996.

[67] C. K. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*. MIT press Cambridge, MA, 2006, vol. 2.

[68] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[69] B. Hurley and B. O'Sullivan, "Statistical Regimes and Runtime Prediction." in *IJCAI*, vol. 15, 2015, pp. 318–324.

[70] A. Kostovska, A. Jankovic, D. Vermetten, J. De Nobel, H. Wang, T. Eftimov, and C. Doerr, "Per-run Algorithm Selection with Warm-Starting Using Trajectory-Based Features," in *Parallel Problem Solving from Nature – PPSN XVII*, G. Rudolph, A. V. Kononova, H. Aguirre, P. Kerschke, G. Ochoa, and T. Tušar, Eds. Cham: Springer International Publishing, 2022, vol. 13398, pp. 46–60.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3703313

[71] A. Kostovska, A. Jankovic, D. Vermetten, S. Džeroski, T. Eftimov, and C. Doerr, "Comparing Algorithm Selection Approaches on Black-Box Optimization Problems," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. Lisbon Portugal: ACM, Jul. 2023, pp. 495–498.

[72] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco California USA: ACM, Aug. 2016, pp. 785–794.

[73] F. Mao, M. Chen, K. Zhong, J. Zeng, and Z. Liang, "An XGBoost-assisted evolutionary algorithm for expensive multiobjective optimization problems," *Information Sciences*, vol. 666, p. 120449, 2024.

[74] G. Zakrzewski, X. Yao, and J. Mańdziuk, "Surrogate management strategies for accelerating parallel algorithm portfolio construction," nov 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17501674

[75] O. Mersmann, B. Bischl, H. Trautmann, M. Wagner, J. Bossek, and F. Neumann, "A novel feature-based approach to characterize algorithm performance for the traveling salesperson problem," *Annals of Mathematics and Artificial Intelligence*, vol. 69, pp. 151–182, 2013.

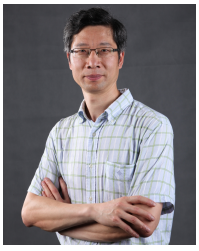
[76] J. Bossek, "{netgen}: Network generator for combinatorial graph problems," 2015.

[77] J. Bossek, P. Kerschke, A. Neumann, M. Wagner, F. Neumann, and H. Trautmann, "Evolving diverse TSP instances by means of novel and creative mutation operators," in *Proceedings of the 15th ACM/SIGEVO Conference on Foundations of Genetic Algorithms*. Potsdam Germany: ACM, Aug. 2019, pp. 58–71.

[78] F. Wilcoxon, "Individual Comparisons by Ranking Methods," in *Breakthroughs in Statistics*, S. Kotz and N. L. Johnson, Eds. New York, NY: Springer New York, 1992, pp. 196–202.



Grzegorz Zakrzewski received the M.Sc. degree in data science from the Warsaw University of Technology, Warsaw, Poland, in 2025, where he is currently pursuing the Ph.D. degree in computer science.
He was a data analyst in technology companies. His current research interests include continuous optimization, meteorology, and forecasting extreme weather phenomena



Xin Yao (Fellow, IEEE) obtained his Ph.D. in 1990 from the University of Science and Technology of China (USTC), Hefei, China; MSc in 1985 from North China Institute of Computing Technologies, Beijing, China; and BSc in 1982 from USTC. He is the Vice President (Research and Innovation) and Tong Tin Sun Chair Professor of Machine Learning at Lingnan University, Hong Kong, China. He is a Fellow of IEEE and Hong Kong Academy of Engineering. He served as the President (2014-15) of IEEE Computational Intelligence Society (CIS) and the Editor-in-Chief (2003-08) of IEEE Transactions on Evolutionary Computation. His major research interests include evolutionary computation, machine learning and AI ethics. His work won the 2001 IEEE Donald G. Fink Prize Paper Award; 2010, 2016 and 2017 IEEE Transactions on Evolutionary Computation Outstanding Paper Awards; 2011 IEEE Transactions on Neural Networks Outstanding Paper Award; 2010 BT Gordon Radley Award for Best Author of Innovation (Finalist); and other best paper awards at conferences. He received the 2012 Royal Society Wolfson Research Merit Award, 2013 IEEE CIS Evolutionary Computation Pioneer Award, and the 2020 IEEE Frank Rosenblatt Award.



Jacek Mańdziuk (Senior Member, IEEE) received his M.Sc. (Honors) and Ph.D. degrees in Applied Mathematics from the Warsaw University of Technology, Warsaw, Poland, in 1989 and 1993, respectively, and a D.Sc. degree in Computer Science from the Polish Academy of Sciences in 2000. In 2011, he was awarded the academic title of Professor by the President of the Republic of Poland. He is a Full Professor at the Faculty of Mathematics and Information Science, Warsaw University of Technology, where he heads the Division of Artificial Intelligence and Computational Methods. He is a Member of the Polish Academy of Sciences and a Senior Member of IEEE. He was a Senior Fulbright Scholar at the University of California, Berkeley (USA) in 1996–1997 and a Visiting Professor at Nanyang Technological University (Singapore) in 2015–2016. Professor Mańdziuk served as General Chair of the 5th Polish Conference on Artificial Intelligence (2024), General Co-Chair of the IEEE Congress on Evolutionary Computation (2021), and Chair of the IEEE SSCI Symposium on Computational Intelligence for Human-like Intelligence (2013–2023). His research interests include the application of artificial and computational intelligence to security and social good, bilevel optimization, abstract visual reasoning, games, human–machine cooperation, and human-like learning and problem-solving.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/